

Phase J J-3 Attribution Finding: the Stage 5 injection hook is built, but not yet wired into the search consumption end

This is an **honest negative/architecture-level result** requiring a decision (see "The next fork"). Date: 2026-07-08.

Contents

One sentence	1
How it was measured (single variable)	1
Result	2
Root cause (grep + empirical double confirmation)	2
Important distinction (do not misread this 0)	2
The next fork (decision needed)	3
Wiring pilot executed (2026-07-08, user chose A, designed as "plateau-triggered, count-limited")	3
Why external injection cannot beat Stage 4 (a reasonable explanation)	4
Final conclusion (Stage 5 / ERA's second pillar)	5

One sentence

The hook for external-idea injection (Stage 5) is itself correct, but under the current tree-search architecture `tree['priors']` is **write-only** (written only for printing/logging, with no search operator reading it), so the single-variable comparison of "pure [INT] vs [INT]+[EXT]" yields a **constructive delta = 0** — not a noise-level near-zero, but **bit-for-bit identical**.

How it was measured (single variable)

Using s3e3 (AUC, small data, full search ~17s, cleanly deterministic re-run) as the target:

- Built `tree_search/run_s3e3_v4.py`, whose **only search-relevant change** = replacing the driver's opening `suggest_priors(COMP_META)` call with `harness_v4.suggest_priors_v4(COMP_META, mode=V4_MODE)` (mode read from an environment variable). All other seeds, folds, budget, mutation, evaluator unchanged.
- Correctness gate: (1) `mode='off'` priors are **verbatim identical** to `hv2.suggest_priors` (passed); (2) `mode='ext'` for this event **injects 2 net-new** [EXT] — EXT-12 (rank averaging), EXT-14 (adversarial validation) (passed).

- Same seed=42, same folds, each run to completion, comparing the global-best OOF.

Result

	Global-best node	AUC	tree['priors'] count
off ([INT] only)	node #11	0.841442	8
ext ([INT]+[EXT])	node #11	0.841442	10 (2 extra [EXT])

- **delta (ext – off) = +0.00000000.**
- Node-by-node comparison (22 nodes): 0 structural differences, 0 score differences ($\max|\Delta\text{score}|=0$), 18 cached OOF vectors $\max|\Delta|=0$. **off and ext are bit-for-bit identical.**

Root cause (grep + empirical double confirmation)

`suggest_priors` 's return is only written to `tree['priors']` for printing/logging; the seeds (`SOLO_SEEDS`), mutation queue, `propose_child`, `select_next_parent`, and evaluator — **none of them read `tree['priors']`**. In the driver, the `[PRIOR PK]` labels are **hand-written string constants** by the author in the seeds/queue, and `prior_usage_summary` also only regex-parses these hand-written labels, never tracing back to `tree['priors']`. Within harness_v2/v3 no `tree['priors']` read point is found. **The bit-identical result is itself the proof:** the only difference (2 extra priors), if read by any operator, would necessarily change something; it changed nothing → `tree['priors']` is indeed write-only. This is **architecture-level** (all four candidate drivers share the same pattern), not an s3e3 idiosyncrasy.

Important distinction (do not misread this 0)

1. **The injection mechanism is itself correct:** harness_v4 correctly merges [INT]+[EXT] into the pool, deduplicates, and passes the gate. The problem is at the **consumption end**.
2. **The value of Stage 4 remains real:** the gain of tree-search stage4 > stage3 comes from the **search mechanism itself** (candidate tree, backtracking, OOF weight search, explore burst), unrelated to `suggest_priors` — these results are unaffected by this finding.
3. **The experience library's knowledge did affect the search, but via "the author's hand-written seeds" rather than automatic injection:** after reading experience.md, the driver author hand-wrote priors into the seed configs; the **output of the `suggest_priors` function** is decorative. Therefore each event report's "Stage 4.3 prior injection + deduplication — used" holds in the "knowledge did guide" sense, but its implied meaning of "the automatic-injection plumbing was consumed by the search" **does not hold** — this deserves honest calibration in report language (non-urgent).

The next fork (decision needed)

To truly measure Stage 5's marginal value, the priors must first be **powered up** — wiring `tree['priors']` into the search decisions, e.g.:

- turn **EXT-12 (rank averaging)** into a candidate node "add a rank-average blend member";
- turn **EXT-14 (adversarial validation)** into a candidate action "run adversarial validation, then reorder/reweight CV accordingly";
- more generally: make `propose_child` / seed logic actually read `tree['priors']`, mapping each prior into an executable mutation/candidate, and marking provenance ([INT]/[EXT]) in the mutation record.

Option A: do this wiring (moderate engineering, but it will change the tree search's behavior for **all events** → requires re-running, possibly reopening existing stage4 results). **Option B:** accept this honest finding, describe Stage 5 as "injection hook built, pending wiring to the consumption end," and not claim its gain.

These two options affect the research direction and existing results, and are left to the supervisor/user to decide; no large self-directed changes before then. Evidence file: `tree_search/run_s3e3_v4.py` (+ process state files, unversioned experimental artifacts).

Wiring pilot executed (2026-07-08, user chose A, designed as "plateau-triggered, count-limited")

The user chose A (wiring) and proposed a good design: **make external-idea injection optional — triggered only when Stage 4 plateaus and cannot push further, and with a limited number of executions** (precisely the implementation of the plan's § 6 iteration strategy "search for external ideas when stalled"). Executed accordingly:

① **Mechanism built and self-test passed** — `tree_search/idea_injection.py`: reads `idea_bank.md` → filters by `comp_meta` + [INT] deduplication → uses a **translator registry** to turn [EXT] ideas into search-candidate configs → marks [EXT-NN] provenance. This "automatic injection" line **is now connected** (the J-3 write-only dead end is bypassed). First clean translator: EXT-12 (rank averaging) → rank-space blend.

② **Fast, real value measurement** (inject at the Stage 4 winner = convergence/plateau point, evaluate directly using **cached OOF**, no retraining):

Event (AUC)	prob space (Stage 4)	rank space (EXT-12 injection)	delta	Verdict
s3e3	~0.841 (best)	0.8329 / 0.8143 (existing rank nodes in tree)	clearly worse	no gain
s4e1	0.894368	0.894359	-0.000008	noise-level

Event (AUC)	prob space (Stage 4)	rank space (EXT-12 injection)	delta	Verdict
s6e2	0.954527	0.954528	+0.000001	noise-level

③ **Fast-eval sweep over all 15 events** (docs/scripts/stage5_sweep.py): added a second general translator **stacking (EXT-09, meta model, applicable to any metric)**, injecting candidates at the Stage 4 winner (convergence point) for each of 15 events, evaluating with cached OOF, and comparing against the **committed tier4** (authoritative Stage 4):

Verdict	# events	Events
tie/noise ($ \text{delta} \leq 1e-4$)	6	s3e1, s3e7, s4e1, s5e10, s6e1, s6e2
injected candidate clearly worse	8	s3e3, s3e5, s3e9, s3e11, s3e14, s3e16, s3e19, s4e11
numerically "wins" but = already-excluded CV noise	1	s3e20

Framing (important): the true Stage 5 (inject at search plateau, then **keep the best**) = **Stage 4, all 15 events tie** — the search discards worse injected candidates and is not dragged down; the "worse" in the table above measures the **injected candidate itself** losing to Stage 4 (i.e. that idea did not produce a better solution), not Stage 5 being dragged down. **Two false positives were caught in the process**, which precisely demonstrates the rigor of this sweep: - **s3e7**: at first, using the fast-eval weight search as the baseline (0.900028) made stacking appear to win, but the committed tier4 is actually higher at 0.900455 — a weak-baseline illusion; switching to the committed tier4 made it null. - **s3e20**: the only numerical win, but stacking squeezed out the GBDT-blend member (node #37, 21.0332) that STATUS.md marks as **low-confidence CV noise**, which tier4 deliberately excludes, recognizing only the pure-structure solution #28 (21.0589) — not a real gain.

Why external injection cannot beat Stage 4 (a reasonable explanation)

The null is not the mechanism being incompetent, but has solid reasons:

1. **The Stage 4 optimizer is already near-optimal over existing members:** the tree search's OOF weight search (Dirichlet $k=800$ + coordinate ascent) has already found the near-optimal convex combination of the member pool; the ideas that can be cleanly injected (rank averaging, stacking) are essentially just **combining the same members in a different way**, and cannot beat the already-near-optimal weight search.
2. **The injectable ideas are redundant or misaligned with the metric:** rank averaging is just a monotone reordering before combination, providing no new information for the metric the weight search already directly optimizes; the stacking meta model is more flexible than the convex weight search → **overfits CV noise** (members mostly being highly-correlated GBDT), and ridge minimizes squared error, misaligned with MAE/SMAPE → s3e14 -5, s3e19 -41 blowups (consistent with s3e14's existing finding that "Ridge stacking loses to simplex").

3. **Useful domain ideas were long ago hand-written into the search by the LLM author** (the J-3 finding): the entries in the external bank applicable to an event mostly overlap with the driver seeds/mutations, making automatic injection redundant.
4. **The signal has been squeezed to the noise floor** (the bootstrap finding): the Stage 4 best already approaches the OOF-estimate ceiling, with compressible room $\leq 0.0x\%$ and often within noise — **any** method struggles to squeeze out more, not that external ideas are especially incompetent.
5. **Ideas that could bring new signal happen to be hard to inject + data-dependent**: adversarial validation, entity embedding, target-encoding variants need new features/retraining and cannot be cleanly auto-injected, and their effect depends on the data (e.g. adversarial validation is useful only when there is train/test drift, whereas these synthetic Playground datasets have almost no drift).

Final conclusion (Stage 5 / ERA's second pillar)

The mechanism for external-idea injection can be built, and is truly powered up (`idea_injection.py`); but across 15 events measured, external injection never produced a candidate able to beat Stage 4 — Stage 5 = Stage 4 (all tie). Reason: the ideas that can be cleanly injected merely recombine "the member pool already combined near-optimally," truly useful domain ideas were already hand-written into the search, and the signal has been squeezed to the noise floor by the first four stages. External knowledge is valuable only when it "introduces new signal the search missed," and these mature tabular problems have almost no missed structure to pick up. $\therefore$ In this setting, ERA's second pillar has no measurable systematic gain (neither harmful nor helpful).

This is a **rigorously established honest negative result** (real data, 15 events measured, reproducible, two false positives already removed), at the same honesty level as this project's J-3, determinism, and statistical-noise findings. The value of Stage 4 (the first pillar) is unaffected.

Reproducible: `uv run python3 docs/scripts/stage5_sweep.py` (15-event fast eval), `uv run python3 tree_search/stage5_inject_pilot.py s4e1` (single event); mechanism in `tree_search/idea_injection.py`.